

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №9
по дисциплине «Вычислительная математика»
Тема: Составные формулы прямоугольников, трапеций, Симпсона.

Студент гр. 4342

Митюков М.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Цель лабораторной работы — изучить численные методы вычисления определённого интеграла с помощью составных квадратурных формул прямоугольников, трапеций и Симпсона на примере функции $f(x)$. В ходе работы необходимо программно реализовать данные методы и алгоритм автоматического выбора числа разбиений n (шага h) с использованием правила Рунге для достижения заданной точности ε . Дополнительной задачей является проведение сравнительного анализа эффективности используемых формул на основе полученных значений интеграла и требуемого числа разбиений отрезка для каждой из них.

Задание.

Повышения точности численного интегрирования добиваются путем применения составных формул. Для этого при нахождении определенного интеграла отрезок $[a, b]$ разбивают на четное $n = 2m$ число отрезков длины $h = (b - a) / n$ и на каждом из отрезков длины $2h$ применяют соответствующую формулу. Таким образом получают составные формулы прямоугольников, трапеций и Симпсона.

На сетке $x_i = a + ih$, $y = f(x_i)$, $i = 0, 1, 2, \dots, 2m$, составные формулы имеют следующий вид:

Формула прямоугольников

$$\int[a; b] f(x) dx = h * \sum_{i=0}^{n-1} f(x_i + h/2) + R_1;$$

Формула трапеций

$$\int[a; b] f(x) dx = (h/2) * \sum_{i=0}^{n-1} (f(x_i) + f(x_{i+1})) + R_2;$$

Формула Симпсона

$$\int[a; b] f(x) dx = (h/3) * \sum_{i=0}^{m-1} (f(x_{2i}) + 4 * f(x_{2i+1}) + f(x_{2i+2})) + R_3,$$

где R_1, R_2, R_3 — остаточные члены. При $n \rightarrow \infty$ приближенные значения интегралов для всех трех формул, в предположении отсутствия погрешностей округления, стремятся к точному значению интеграла [1, 7, 8].

Для практической оценки погрешности квадратурной формулы можно использовать правило Рунге. Для этого проводят вычисления на сетках с шагом h и $h/2$, получают приближенные значения интеграла I_h и $I_{h/2}$ и за окончательные значения интеграла принимают величины:

$$I = I_{h/2} + (I_{h/2} - I_h) / 3 \text{ — для формулы прямоугольников;}$$

$$I = I_{h/2} + (I_{h/2} - I_h) / 3 \text{ — для формулы трапеций;}$$

$$I = I_{h/2} + (I_{h/2} - I_h) / 15 \text{ — для формулы Симпсона.}$$

За погрешность приближенного значения интеграла для формул прямоугольников и трапеций тогда принимают величину: $|I_{h/2} - I_h| / 3$,

а для формулы Симпсона: $|I_{h/2} - I_h| / 15$.

В лабораторной работе №6 требуется, используя квадратурные формулы прямоугольников, трапеций и Симпсона, вычислить значения заданного интеграла и, применив правило Рунге, найти наименьшее значение n , то есть наибольшее значение шага h , при котором каждая из указанных формул дает приближенное значение интеграла с погрешностью ε , не превышающей заданную.

Порядок выполнения лабораторной работы №6.

- 1) Составить программы-функции для вычисления интегралов по формулам прямоугольников, трапеций и Симпсона.
- 2) Составить программу-функцию для вычисления подынтегральной функции.
- 3) Составить главную программу, содержащую оценку по Рунге погрешности каждой из перечисленных выше квадратурных формул, удваивающих n до тех пор, пока погрешность не станет меньше ε , и осуществляющих печать результатов: значения интеграла и значения n для каждой формулы.
- 4) Провести вычисления по программе, добиваясь, чтобы результат удовлетворял требуемой точности.
- 5) Результаты работы оформить в виде краткого отчета, содержащего сравнительную оценку применяемых для вычисления формул.

Варианты заданий приведены в таблице 4.1. $\varepsilon = 0.01; 0.001; 0.0001$.

Вариант 13: $f(x) = \int_{\pi/2}^{\pi} \sqrt{x} \exp(-x^2) dx$

Выполнение работы.

1. Составить программы-функции для вычисления интегралов по формулам прямоугольников, трапеций и Симпсона.

Для решения задачи были реализованы три функции, вычисляющие определенный интеграл на отрезке $[a, b]$, разбитом на n равных элементарных отрезков длиной $h = (b - a) / n$. Каждая функция реализует свой математический подход к аппроксимации площади под кривой.

Метод прямоугольников (модификация средних точек):

В данной функции отрезок интегрирования делится на n частей. Значение интеграла вычисляется как сумма площадей прямоугольников, высота которых равна значению функции в средней точке каждого элементарного отрезка. Этот метод точнее, чем использование левых или правых границ.

Алгоритм в коде реализует следующую формулу:

$$I = h * (f(x_0 + h/2) + f(x_1 + h/2) + \dots + f(x_{n-1} + h/2))$$

где $x_i = a + i * h$. Программная реализация использует цикл для накопления суммы значений функции, которая в конце умножается на шаг h .

Метод трапеций:

Функция аппроксимирует график подынтегральной функции ломаной линией. Площадь на каждом шаге вычисляется как площадь прямоугольной трапеции. Для оптимизации вычислений значения функции во внутренних узлах суммируются дважды, а на краях — один раз.

Математический вид, реализованный в программе:

$$I = h * ((f(a) + f(b)) / 2 + f(x_1) + f(x_2) + \dots + f(x_{n-1}))$$

В коде это реализовано через инициализацию суммы полусуммой значений на границах и последующее добавление всех промежуточных значений $f(x_i)$ в цикле.

Метод Симпсона (метод парабол):

Это наиболее точный из представленных методов. Вместо прямых линий он использует параболическую интерполяцию по трем точкам на каждом удвоенном шаге. Для работы метода требуется четное количество разбиений n .

Формула, заложенная в программу:

$$I = (h / 3) * (f(a) + f(b) + 4 * S1 + 2 * S2)$$

где $S1$ — сумма значений функции в узлах с нечетными индексами ($x_1, x_3 \dots$), а $S2$ — сумма значений в узлах с четными индексами ($x_2, x_4 \dots$). В коде используется проверка индекса на четность ($i \% 2$) для выбора соответствующего коэффициента (4 или 2).

2. Составить программу-функцию для вычисления подынтегральной функции

В данном пункте была разработана вспомогательная функция $f(x)$, которая возвращает значение подынтегрального выражения для любого заданного значения аргумента x . Выделение вычислений в отдельную функцию позволяет основной программе и методам интегрирования обращаться к математической модели задачи в едином формате.

Согласно индивидуальному варианту, подынтегральная функция имеет вид: $f(x) = \sqrt{x} * \exp(-x^2)$

Особенности программной реализации:

Функция объявлена с типом `double`, что необходимо для поддержания высокой точности вычислений (64-битное число с плавающей запятой).

Для извлечения квадратного корня используется стандартная функция `sqrt(x)` из заголовочного файла `cmath`.

Вычисление экспоненты (числа « e » в заданной степени) реализовано с помощью функции `exp()`.

Математическое выражение в степени $-x^2$ представлено в коде как $-(x * x)$. Такой подход является более производительным и точным для второй степени, чем использование универсальной функции возведения в степень `pow(x, 2)`.

Эта функция вызывается многократно внутри циклов интегрирования для определения значений $y = f(x_i)$ в узлах расчетной сетки на интервале от $p_i / 2$ до p_i .

3. Составить главную программу, содержащую оценку по Рунге погрешности каждой из перечисленных выше квадратурных формул, удваивающих n до тех пор, пока погрешность не станет меньше ε , и осуществляющих печать результатов: значения интеграла и значения n для каждой формулы

Главная программа координирует работу всех функций и реализует итерационный процесс достижения заданной точности. Основная логика заключается в последовательном измельчении шага интегрирования до тех пор, пока расчетная погрешность не станет допустимой.

Основные этапы работы главной программы:

Инициализация параметров:

Задаются границы интегрирования $a = p_i / 2$ и $b = p_i$, а также массив требуемых значений точности eps (0.01, 0.001, 0.0001). Для каждого значения точности запускается цикл расчетов.

Итерационный цикл (правило Рунге):

Для каждого метода интегрирования программа начинает расчет с небольшого числа разбиений (например, $n = 2$). На каждой итерации:

Вычисляется значение интеграла I_h при текущем шаге h .

Число разбиений n удваивается, и вычисляется новое значение интеграла $I_{h/2}$ с половинным шагом.

Рассчитывается абсолютная погрешность по правилу Рунге.

Для методов прямоугольников и трапеций (второй порядок точности):

$$\text{Error} = \text{abs}(I_{h/2} - I_h) / 3$$

Для метода Симпсона (четвертый порядок точности):

$$\text{Error} = \text{abs}(I_{h/2} - I_h) / 15$$

Если полученная погрешность $Error$ больше заданного значения eps , процесс повторяется: текущее $I_{h/2}$ принимается за старое, а n снова удваивается.

Уточнение результата:

После выхода из цикла программа вычисляет окончательное (уточненное) значение интеграла по формуле: $I_{final} = I_{h/2} + (I_{h/2} - I_h) / (2^p - 1)$ где p — порядок точности метода (2 или 4). Это позволяет получить результат, более близкий к истинному значению, чем просто расчет на мелкой сетке.

Печать результатов:

Для каждого метода программа выводит в консоль итоговое количество разбиений n , при котором была достигнута точность, само значение интеграла и фактическую погрешность. Вывод форматируется с помощью манипуляторов `setw` и `setprecision` для обеспечения наглядности и сравнения данных.

4. Провести вычисления по программе, добиваясь, чтобы результат удовлетворял требуемой точности

На четвертом этапе были проведены вычисления по составленной программе. Для заданного интеграла функции $f(x) = \sqrt{x} * \exp(-x^2)$ на отрезке от $\pi/2$ до π были использованы три квадратурные формулы: формула прямоугольников (средних), формула трапеций и формула Симпсона.

Вычисления проводились для трех значений точности: $eps = 0.01$, $eps = 0.001$, $eps = 0.0001$.

Для каждой формулы программа начинала вычисления с минимального количества разбиений $n = 2$, затем последовательно удваивала это значение. На каждом шаге интеграл вычислялся дважды: сначала при текущем значении n , затем при удвоенном количестве разбиений. После этого по правилу Рунге оценивалась погрешность результата:

Для методов прямоугольников и трапеций: $Error = \text{abs}(I_{h2} - I_h) / 3$

Для метода Симпсона: $Error = \text{abs}(I_{h2} - I_h) / 15$

Если оценка погрешности была больше заданного значения ϵ_{ps} , количество разбиений снова увеличивалось в два раза. Процесс продолжался до тех пор, пока оценка погрешности не становилась меньше заданной точности.

В результате были получены следующие значения:

ϵ_{ps}	Метод	n	h	Значение интеграла	Оценка погрешности
0.01	Прямоугольники	4	0.392699	0.02946727	0.00183
0.01	Трапеции	4	0.392699	0.03521477	0.00375
0.01	Симпсон	4	0.392699	0.03146842	0.000175
0.001	Прямоугольники	8	0.196350	0.03089906	0.000477
0.001	Трапеции	8	0.196350	0.03234102	0.000958
0.001	Симпсон	4	0.392699	0.03146842	0.000175
0.0001	Прямоугольники	32	0.049087	0.03134945	0.000030
0.0001	Трапеции	32	0.049087	0.03143966	0.000060
0.0001	Симпсон	8	0.196350	0.03138310	0.000005

По результатам вычислений видно, что все три метода позволяют получить приближенное значение интеграла с заданной точностью. При этом формула Симпсона достигает требуемой точности при значительно меньшем количестве разбиений n . Это объясняется тем, что формула Симпсона имеет четвертый порядок точности, в то время как формулы прямоугольников и трапеций — только второй.

Для наиболее строгой точности $\epsilon_{ps} = 0.0001$ методы прямоугольников и трапеций потребовали $n = 32$, в то время как формуле Симпсона оказалось достаточно $n = 8$ для получения результата с запасом по точности. Таким образом, в данной задаче формула Симпсона показала себя как наиболее эффективная и экономичная с точки зрения вычислительных затрат.

5. Результаты работы оформить в виде краткого отчета, содержащего сравнительную оценку применяемых для вычисления формул.

В ходе выполнения лабораторной работы было проведено численное вычисление интеграла $\int$ от $\pi/2$ до π $\sqrt{x} * \exp(-x^2) dx$ с использованием трех составных квадратурных формул: формулы прямоугольников, формулы трапеций и формулы Симпсона. Для оценки точности результатов применялось правило Рунге. Расчеты выполнялись для трех значений требуемой точности: $\text{eps} = 0.01$, $\text{eps} = 0.001$ и $\text{eps} = 0.0001$.

По полученным данным видно, что все три метода позволяют найти значение интеграла с заданной точностью, однако эффективность методов различается. Главным критерием сравнения является количество разбиений n , которое потребовалось для достижения заданной погрешности. Чем меньше значение n , тем меньше вычислений нужно выполнить программе, следовательно, тем эффективнее метод.

Для точности $\text{eps} = 0.01$ все три метода достигли нужной точности уже при $n = 4$. Однако оценки погрешности у методов заметно отличаются. У формулы прямоугольников погрешность составила 0.00183354, у формулы трапеций — 0.00375150, а у формулы Симпсона — всего 0.00017515. Это показывает, что даже при одинаковом числе разбиений формула Симпсона дает значительно более точный результат.

Для точности $\text{eps} = 0.001$ различие между методами становится более заметным. Формуле прямоугольников и формуле трапеций потребовалось увеличить число разбиений до $n = 8$, то есть выполнить два удвоения n . При этом формула Симпсона снова достигла требуемой точности уже при $n = 4$. Оценка погрешности Симпсона осталась равной 0.00017515, что намного меньше заданного значения 0.001.

Для наиболее строгой точности $\text{eps} = 0.0001$ формулы прямоугольников и трапеций потребовали уже $n = 32$. Это означает, что программе пришлось выполнить четыре удвоения количества разбиений относительно начального шага. Формула Симпсона при этом удовлетворила требуемой точности при $n = 8$. Это является главным подтверждением того, что формула Симпсона в данной задаче оказалась наиболее эффективной.

Сравнение значений n можно представить следующим образом:

Точность ϵ_{rs}	Прямоугольники	Трапеции	Симпсон
0.01	$n = 4$	$n = 4$	$n = 4$
0.001	$n = 8$	$n = 8$	$n = 4$
0.0001	$n = 32$	$n = 32$	$n = 8$

Из таблицы видно, что при ужесточении требования к точности методы прямоугольников и трапеций требуют значительного увеличения числа разбиений. При переходе от $\epsilon_{rs} = 0.01$ к $\epsilon_{rs} = 0.0001$ количество разбиений для этих методов выросло с 4 до 32. У формулы Симпсона число разбиений выросло всего до 8.

Также можно сравнить оценки погрешности при одинаковом значении $n = 4$ для точности $\epsilon_{rs} = 0.01$:

Метод	n	Оценка погрешности
Прямоугольники	4	0.00183354
Трапеции	4	0.00375150
Симпсон	4	0.00017515

По этим данным видно, что формула Симпсона дает ошибку примерно в 10–20 раз меньше, чем формула прямоугольников и формула трапеций. Это связано с тем, что формула Симпсона имеет более высокий порядок точности. Формулы прямоугольников и трапеций имеют второй порядок точности, а формула Симпсона — четвертый порядок точности. Поэтому при уменьшении шага сетки результат по формуле Симпсона уточняется значительно быстрее.

Интересно также сравнить сами приближенные значения интеграла. При самой строгой точности $\epsilon_{rs} = 0.0001$ были получены следующие уточненные значения по Рунге:

Метод	I с уточнением по Рунге
Прямоугольники	0.03137951
Трапеции	0.03137953
Симпсон	0.03137741

Значения, полученные по формулам прямоугольников и трапеций при $n = 32$, практически совпадают между собой. Это говорит о том, что оба метода при достаточно мелком разбиении сходятся к одному и тому же значению интеграла.

Формула Симпсона уже при $n = 8$ дает значение, максимально близкое к этим результатам, причем ее оценка погрешности также удовлетворяет заданному условию с большим запасом.

Формула прямоугольников в данной задаче дает значение интеграла, которое при грубом разбиении получается меньше истинного. Например, при $n = 4$ значение по формуле равно 0.02946727, а уточненное по Рунге — 0.03129778. Формула трапеций, наоборот, дает значение большее: 0.03521477, а после уточнения по Рунге получается 0.03146842. То есть методы прямоугольников и трапеций приближаются к результату с разных сторон, что хорошо видно по численным данным.

Формула Симпсона показывает наилучшее поведение: уже при $n = 4$ она дает значение 0.03146842, а уточнение по Рунге дает 0.03129327. При этом оценка погрешности при $n = 8$ составляет всего 0.00000569, что на порядок меньше самого строгого значения $\epsilon_{ps} = 0.0001$.

Таким образом, по результатам вычислений можно сделать вывод, что все рассмотренные квадратурные формулы применимы для вычисления данного интеграла, но их эффективность различна. Формула трапеций и формула прямоугольников требуют кратного увеличения числа разбиений при повышении точности. Формула Симпсона обеспечивает наибольшую точность при минимальном количестве разбиений. Поэтому для данного интеграла наиболее эффективной оказалась составная формула Симпсона.

Вывод.

В ходе лабораторной работы были изучены составные квадратурные формулы для численного вычисления определенного интеграла на примере интеграла от $\pi/2$ до π функции $\sqrt{x} * \exp(-x^2) dx$. Были реализованы подпрограмма вычисления подынтегральной функции $f(x) = \sqrt{x} * \exp(-x^2)$, составная формула прямоугольников, составная формула трапеций и составная формула Симпсона. Для практической оценки точности было применено правило Рунге: интеграл вычислялся на сетках с шагами h и $h/2$, после чего по разности полученных значений определялась оценка погрешности. Проведены вычисления для значений $\epsilon = 0.01$, $\epsilon = 0.001$ и $\epsilon = 0.0001$, при этом количество разбиений n автоматически удваивалось до тех пор, пока оценка погрешности не становилась меньше заданной точности. По результатам вычислений установлено, что все три формулы позволяют получить значение интеграла с требуемой точностью, однако их эффективность различается. Формулы прямоугольников и трапеций при уменьшении ϵ до 0.0001 потребовали увеличения числа разбиений до $n = 32$, тогда как формула Симпсона достигла этой точности уже при $n = 8$. Это подтвердило более высокую точность формулы Симпсона и показало, что для данного интеграла она является наиболее эффективной из рассмотренных квадратурных формул.

ПРИЛОЖЕНИЕ А

Название файла: main.cpp

```
#include <iostream>
#include <cmath>
#include <iomanip>
#include <string>

using namespace std;

// Константа Пи (на случай, если M_PI не поддерживается компилятором)
const double PI = acos(-1.0);

// 1) Программа-функция для вычисления подынтегральной функции.
// В вашем варианте: f(x) = sqrt(x) * exp(-x^2)
double f(double x) {
    return sqrt(x) * exp(-x * x);
}

// 2) Программы-функции для вычисления интегралов

// Формула средних прямоугольников
double rectangleRule(double a, double b, int n) {
    double h = (b - a) / n;
    double sum = 0.0;
    for (int i = 0; i < n; ++i) {
        // Берем значение функции в середине каждого отрезка
        double x_mid = a + i * h + h / 2.0;
        sum += f(x_mid);
    }
    return h * sum;
}

// Формула трапеций
double trapezoidalRule(double a, double b, int n) {
    double h = (b - a) / n;
    // Значения на концах отрезка интегрирования делятся пополам
    double sum = (f(a) + f(b)) / 2.0;
    for (int i = 1; i < n; ++i) {
        sum += f(a + i * h);
    }
    return h * sum;
}

// Формула Симпсона (парабол)
// Примечание: n всегда должно быть четным, что гарантируется нашей
// головной программой
double simpsonRule(double a, double b, int n) {
    double h = (b - a) / n;
    double sum = f(a) + f(b); // Крайние точки

    for (int i = 1; i < n; ++i) {
        // Для нечетных индексов коэффициент 4, для четных - 2
        int factor = (i % 2 == 0) ? 2 : 4;
        sum += factor * f(a + i * h);
    }
}
```

```

    return h * sum / 3.0;
}

// 3) Головная функция-помощник: оценка по Рунге и поиск нужного n

void calculateWithRunge(const string& methodName,
                        double (*method)(double, double, int),
                        double a, double b, double eps, int rungeDivisor)
{
    int n = 2; // Начинаем всегда с n=2 (минимальное четное число для
Симпсона)
    double I_h = method(a, b, n); // Интеграл с шагом h
    double I_h2; // Интеграл с шагом h/2 (удвоенное
число разбиений)
    double error; // Абсолютная погрешность по Рунге
    double rungeTerm; // Само слагаемое Рунге (с учетом
знака)

    // Удваиваем n до тех пор, пока погрешность не станет меньше epsilon
    do {
        n *= 2; // Переходим от сетки h к сетки h/2
        I_h2 = method(a, b, n);

        // Вычисляем остаточный член по правилу Рунге:  $(I_{h/2} - I_h) / (2^p - 1)$ 
        rungeTerm = (I_h2 - I_h) / rungeDivisor;
        error = abs(rungeTerm); // Для проверки условия точности берем по
модулю

        I_h = I_h2; // Сохраняем значение I_h/2 для следующей итерации
(он станет I_h)
    } while (error > eps);

    // Вычисляем уточненное значение интеграла (экстраполяция Ричардсона)
    double refined_I = I_h2 + rungeTerm;

    // 4) Печать результатов: значение интеграла и значение n
    cout << left << setw(18) << methodName
        << " | n = " << setw(5) << n
        << " | I_h/2 = " << fixed << setprecision(8) << I_h2
        << " | Уточн. I = " << refined_I
        << " | Погрешность: " << scientific << setprecision(2) << error
    << fixed << endl;
}

// Точка входа в программу

int main() {
    // Границы интегрирования
    double a = PI / 2.0;
    double b = PI;

    // Заданные в варианте значения требуемой точности
    double eps_values[] = {0.01, 0.001, 0.0001};

    cout << "Интегрирование функции f(x) = sqrt(x) * exp(-x^2)\n";
    cout << "На отрезке [pi/2, pi]\n";
    cout << string(95, '=') << endl;
}

```

```

    // Цикл по всем значениям требуемой точности epsilon
    for (double eps : eps_values) {
        cout << "ТРЕБУЕМАЯ ТОЧНОСТЬ (Epsilon) = " << defaultfloat << eps
<< endl;
        cout << string(95, '-') << endl;

        // Для прямоугольников и трапеций делитель в правиле Рунге равен
3 (т.к. порядок точности  $p=2$ ,  $2^2 - 1 = 3$ )
        calculateWithRunge("Прямоугольников", rectangleRule, a, b, eps,
3);
        calculateWithRunge("Трапеций", trapezoidalRule, a, b, eps, 3);

        // Для формулы Симпсона делитель равен 15 (т.к. порядок точности
p=4,  $2^4 - 1 = 15$ )
        calculateWithRunge("Симпсона", simpsonRule, a, b, eps, 15);

        cout << string(95, '=') << endl;
    }

    return 0;
}

```